

ScamShield

AI-Powered Detection of Fake Job Postings
and Recruitment Scams



SF

Theme	Fraud detection for online job postings & recruitment scams
Approach	Hybrid rule-based engine + AI/ML classifier producing an explainable risk score
Contents	Problem Understanding, Proposed Solution, System Architecture, Key Features, Technology Stack, Implementation Plan

1 Problem Understanding

Online job portals, professional social networks, and messaging platforms have made job searching faster and more accessible, but the same openness is exploited by scammers who post fraudulent vacancies. These postings frequently imitate legitimate, well-known companies and use tactics such as unrealistically high salaries, urgent or limited-time hiring language, vague job responsibilities, and requests for upfront payments or sensitive personal and banking information. Victims may lose money, have their identity compromised, or be drawn into fake interview processes designed purely to harvest personal data.

Because fraudulent postings are deliberately designed to resemble genuine ones, manual identification is unreliable and does not scale across the thousands of listings published daily. There is a clear need for an automated, explainable system that can screen job postings at the point of publication or application and warn users before they engage with a potentially fraudulent recruiter.

Core Challenges

- **Scale:** Thousands of postings are published daily across portals, social media, and messaging apps — manual review is impossible.
- **Sophistication:** Scammers copy real company branding, logos, and job descriptions, making postings look authentic.
- **Data sparsity:** Many postings contain limited text, making pure NLP classification difficult without supporting signals.
- **Evolving tactics:** Fraud patterns change constantly, so static keyword lists alone become outdated quickly.
- **User trust and awareness:** Job seekers, especially first-time applicants, may not recognise common red flags on their own.

Objectives

- Automatically analyse job postings from text, metadata, and contact details to detect fraud indicators.
- Combine rule-based heuristics with AI/ML classification for both precision and adaptability.
- Generate a quantifiable, easy-to-understand risk score for every posting.
- Explain *why* a posting is flagged, so users learn to recognise scams themselves.
- Support browser, portal, and API integration so the system can be embedded wherever job seekers browse listings.

2 Proposed Solution

We propose **ScamShield**, a hybrid fraud-detection system that evaluates job postings using two complementary layers: a transparent **rule-based engine** that captures well-known scam patterns, and a **machine-learning classifier** that learns subtler, harder-to-articulate patterns from historical genuine and fraudulent postings. The outputs of both layers are combined into a single **0–100 risk score**, mapped to a Low / Medium / High risk label, along with a plain-language explanation of the contributing factors.

How It Works

- **Ingestion:** A posting is submitted via browser extension, API, upload, or portal integration.
- **Preprocessing:** Text is cleaned and normalised; structured fields (salary, company, contact, location) are extracted.
- **Rule-based screening:** Deterministic checks flag known red flags (e.g., payment requests, personal email domains, salary far above market rate).
- **ML classification:** A trained model estimates the probability that the posting is fraudulent, based on text and metadata features.
- **Company & contact verification:** Company name, domain, and registration details are cross-checked against trusted sources.
- **Score fusion & explanation:** Rule and model outputs are combined into a final score with a human-readable justification.
- **User feedback loop:** Users can confirm or dispute a flag, and confirmed scams are fed back to retrain the model over time.

Why a Hybrid Approach

Approach	Strength	Limitation	Role in ScamShield
Rule-based engine	Transparent, fast, catches known patterns immediately	Cannot adapt to new/unseen scam wording on its own	First-pass filter and explainability source
AI/ML classifier	Learns subtle, evolving patterns from data; generalises well	Needs quality training data; less transparent alone	Deep pattern detection and probability scoring
Verification layer	Confirms real-world legitimacy of company/contact	Depends on external data availability	Cross-check for company and recruiter authenticity

Illustrative Risk Scoring Logic

Each posting accumulates weighted risk points from rule violations and the ML model's fraud probability. The combined score determines the final classification band shown below.

Risk Score	Classification	System Action
0 – 30	Low Risk (Likely Genuine)	Posting shown normally with a green trust indicator
31 – 65	Medium Risk (Caution)	Yellow warning banner with specific caution points shown
66 – 100	High Risk (Likely Scam)	Red alert, posting flagged/hidden, explanation and reporting option shown

3 System Architecture

ScamShield follows a modular, layered architecture so that the rule engine, ML pipeline, and verification services can be maintained, scaled, and retrained independently.

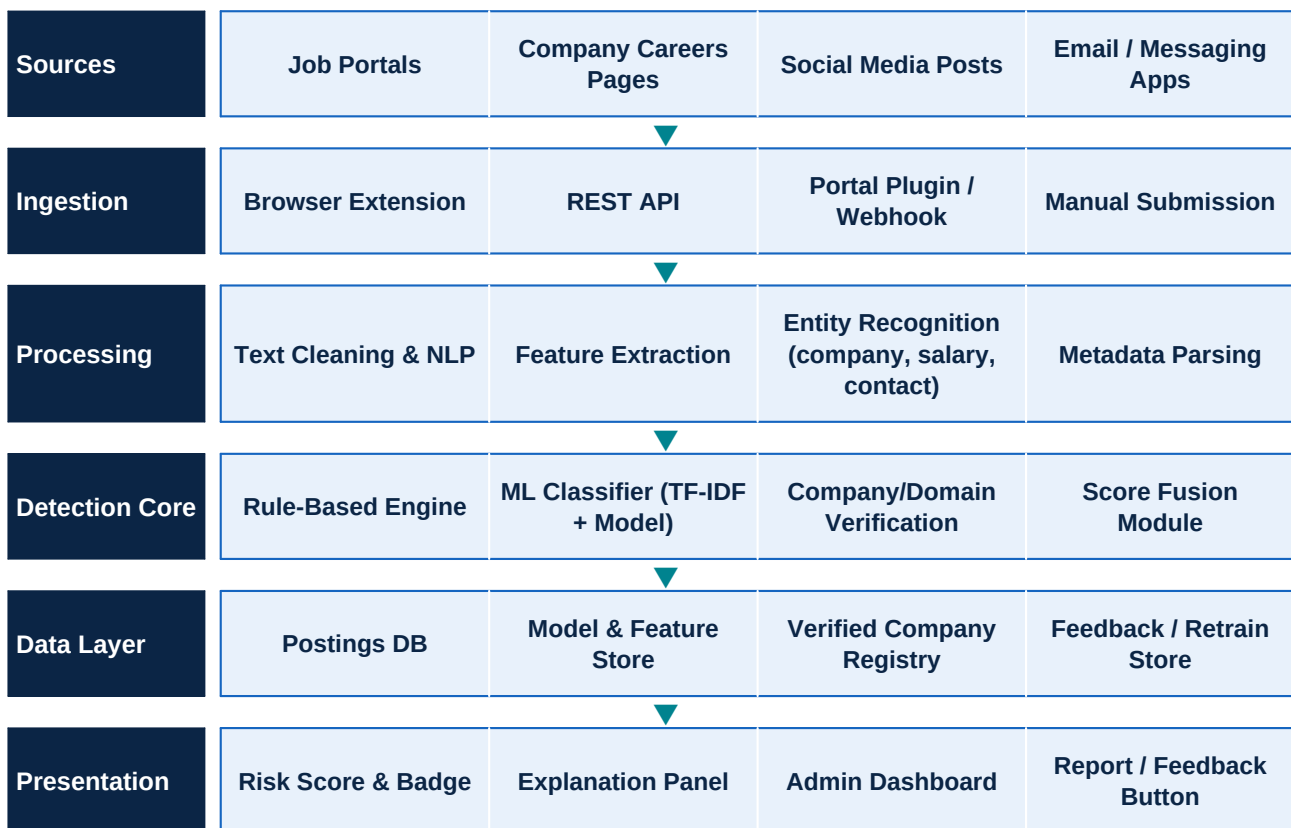


Figure 1: Layered architecture — postings flow from source platforms through ingestion, NLP processing, the hybrid detection core, and back to the user as a scored, explained result.

Component Responsibilities

Component	Responsibility
Ingestion Layer	Collects postings from portals, extension, API, or manual submission and queues them for processing
NLP Preprocessing	Cleans text, tokenises, removes noise, and extracts structured fields (salary, title, location, contact)
Rule Engine	Applies deterministic checks: payment requests, personal-domain emails, salary anomalies, generic descriptions
ML Classifier	Scores posting-fraud probability using text embeddings and metadata features from a trained model
Verification Service	Validates company name, website domain age, and registration against trusted external sources
Score Fusion Module	Combines rule flags and ML probability into a single weighted 0–100 risk score with reasons
Explanation Engine	Converts triggered rules and top model features into plain-language justification text

Component	Responsibility
Dashboard / API	Surfaces results to end users and administrators; exposes REST endpoints for portal integration
Feedback & Retraining	Captures user-confirmed outcomes to periodically retrain and improve the ML model

4 Key Features

1	Automated Risk Scoring Every posting receives a transparent 0–100 risk score and a Low / Medium / High label generated within seconds of submission.
2	Explainable Flags Each flagged posting lists the exact reasons — e.g., "requests upfront payment", "salary 3x above market average", "recruiter uses a free email domain" — so users understand the decision.
3	Hybrid Detection Engine Combines rule-based heuristics for known scam patterns with an ML classifier that adapts to new and evolving fraud language.
4	Company & Contact Verification Cross-checks company names, website domains, and recruiter contact details against trusted registries and domain-age data.
5	Suspicious Keyword & Pattern Detection Identifies urgency language, unrealistic salary claims, vague job descriptions, and grammar/formatting inconsistencies.
6	Payment & Data-Request Detection Specifically flags any request for registration fees, security deposits, bank details, or government ID numbers early in the process.
7	Real-Time Browser & Portal Integration A lightweight extension/plugin can scan postings as users browse job portals or social media, showing an inline risk badge.
8	User Reporting & Feedback Loop Users can confirm or dispute a flag; confirmed scams enrich the training dataset for continuous model improvement.
9	Admin & Analytics Dashboard Gives platform administrators visibility into flagged postings, trends, and false-positive/negative rates.
10	Awareness & Education Layer Displays short, contextual tips (e.g., "Legitimate employers never ask you to pay to get hired") alongside each flag.

5 Technology Stack

The stack favours well-supported, production-proven tools so the system can move from prototype to a scalable service without re-architecture.

Layer	Technology	Purpose
Frontend / Extension	React.js, Chrome/Edge Extension APIs, HTML/CSS/JS	User dashboard, inline risk badges on job portals, alerts UI
Backend API	Python (FastAPI) or Node.js (Express)	REST endpoints for scoring, verification, feedback, and admin operations
NLP & Feature Extraction	spaCy, NLTK, TF-IDF / scikit-learn Vectorizers	Text cleaning, entity extraction, feature engineering
Machine Learning	scikit-learn (Logistic Regression, Random Forest, XGBoost)	Fraud-probability classification; can extend to transformer-based models
Rule Engine	Python rule modules / JSON-configurable rule sets	Deterministic checks for known scam indicators, easily updatable
Verification Services	WHOIS/domain-age APIs, company registry APIs, email validation libraries	Confirms recruiter/company legitimacy
Database	PostgreSQL (structured data), MongoDB (raw postings/logs)	Stores postings, scores, user feedback, and model metadata
Caching & Queue	Redis, Celery / RabbitMQ	Speeds up repeated lookups and manages asynchronous scoring jobs
Model Serving	FastAPI microservice / Flask, Docker containers	Serves the ML model independently for scalability
Deployment & Infra	Docker, Kubernetes (optional), Render / AWS / Azure	Containerised deployment with horizontal scaling
Monitoring & CI/CD	GitHub Actions, Prometheus/Grafana (optional)	Automated testing/deployment and system health monitoring
Security	OAuth2/JWT auth, HTTPS/TLS, input sanitisation	Protects API access and user-submitted data

6 Implementation Plan

The project is planned across six phases, moving from research and data collection to a deployed, continuously-improving system.

Phase	Milestone	Duration	Key Activities
Phase 1	Research & Requirement Analysis	1–2 weeks	Study real scam-posting patterns, define red-flag categories, finalise scope and success metrics.
Phase 2	Data Collection & Preparation	2–3 weeks	Gather labelled genuine/fraudulent postings (public datasets + curated examples), clean and annotate data.
Phase 3	Rule Engine & Feature Engineering	2 weeks	Build the deterministic rule set (payment requests, salary anomalies, contact-domain checks) and extract ML features.
Phase 4	ML Model Development	2–3 weeks	Train and evaluate classifiers (Logistic Regression, Random Forest, XGBoost); tune for precision/recall balance.
Phase 5	System Integration & Score Fusion	2 weeks	Build the API layer, integrate rule engine + ML outputs into the unified risk-scoring module, add explanation generation.
Phase 6	Frontend, Testing & Deployment	2–3 weeks	Develop dashboard/extension UI, conduct unit/integration/user-acceptance testing, deploy via Docker to cloud, set up monitoring.
Phase 7	Feedback Loop & Continuous Improvement	Ongoing	Collect user feedback on flagged postings, periodically retrain the model, and update rules for emerging scam patterns.

Success Metrics

- **Detection accuracy:** Precision and recall of the classifier on held-out fraudulent vs. genuine postings.
- **False-positive rate:** Minimise incorrectly flagging legitimate employers to preserve user trust.
- **Response time:** Risk score generated within a few seconds of submission for real-time browsing use.
- **User trust & adoption:** Growth in extension installs / portal integrations and positive feedback ratings.
- **Scam reports prevented:** Reduction in user-reported financial loss or data-sharing incidents over time.

Risks & Mitigations

Risk	Mitigation
Limited labelled fraud data	Use public phishing/fraud-posting datasets, synthetic augmentation, and active learning from user feedback
Evolving scam tactics reduce accuracy	Scheduled retraining cadence and easily editable, versioned rule configuration
False positives harm genuine employers	Human review queue for medium-risk postings before public flagging; appeal/feedback mechanism

Risk	Mitigation
Verification API rate limits/cost	Cache verification results and fall back gracefully to rule/ML-only scoring when unavailable